ECS 152A Discussion

Week 2

Network Performance

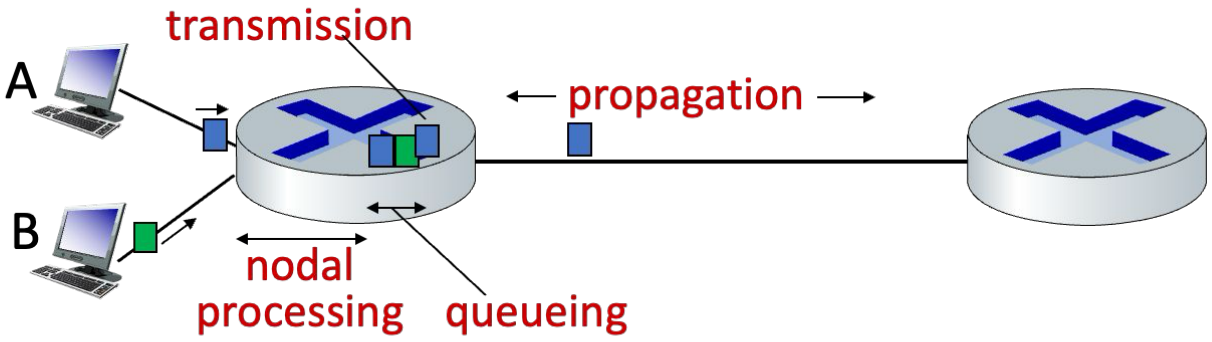
Delays

A packet suffers different types of delay when it is being transmitted across the network.

Types of Delays:

- Processing delay
- Queuing delay
- Transmission delay
- Propagation delay

Packet delay: four sources



$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

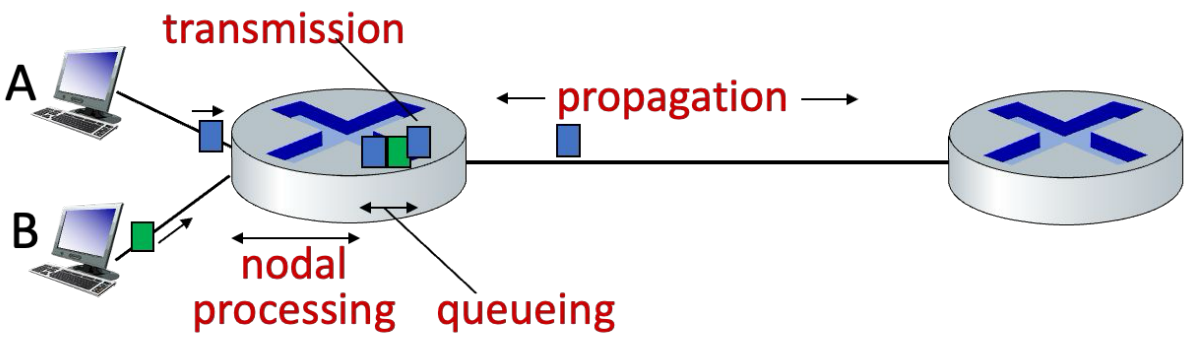
d_{proc}: nodal processing

- check bit errors
- determine output link
- typically < microsecs

d_{queue}: queueing delay

- time waiting at output link for transmission
- depends on congestion level of router

Packet delay: four sources



$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

d_{trans}: transmission delay:

- L: packet length (bits)
- R: link *transmission rate (bps)*
- $d_{trans} = L/R$

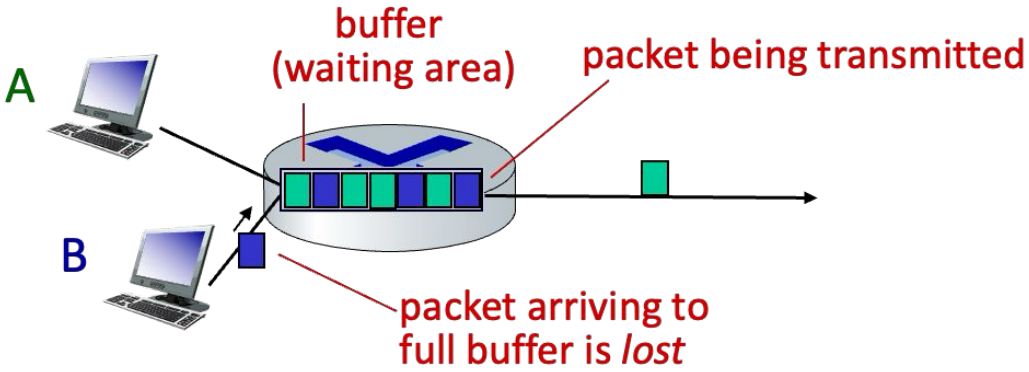
d_{prop}: propagation delay:

- d: length of physical link
- s: propagation speed (~2×10^8 m/sec)
- $d_{prop} = d/s$

d_{trans} and d_{prop} *very* different

Packet loss

- queue (aka buffer) preceding link in buffer has finite capacity
- packet arriving to full queue dropped (aka lost)
- lost packet may be retransmitted by previous node, by source end system, or not at all



Packet loss continued

A - Average Arrival rate

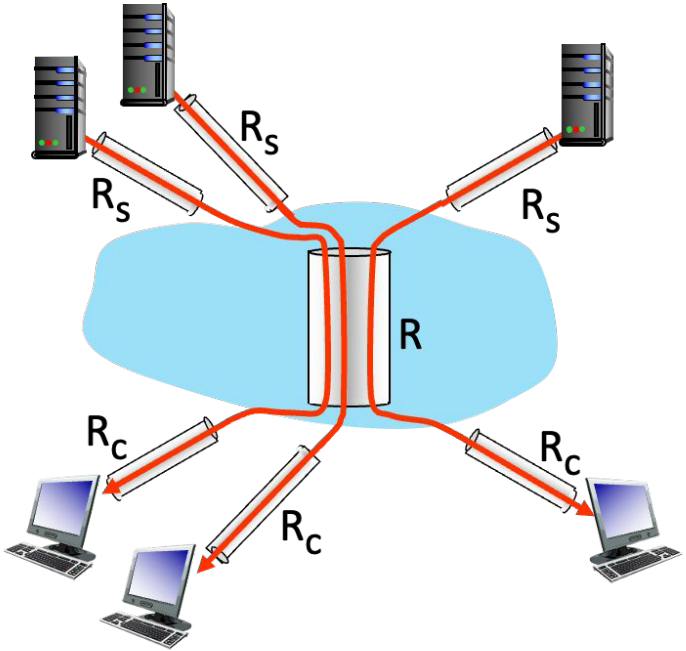
R - Transmission rate

L - Packet length

Ideal case: $(L/R)>A$

Throughput: network scenario

* Check out the online interactive exercises for more examples



10 connections (fairly) share backbone bottleneck link *R* bits/sec

- per-connection end-end throughput: $min(R_c, R_s, R/10)$
- in practice: R_c or R_s is often bottleneck

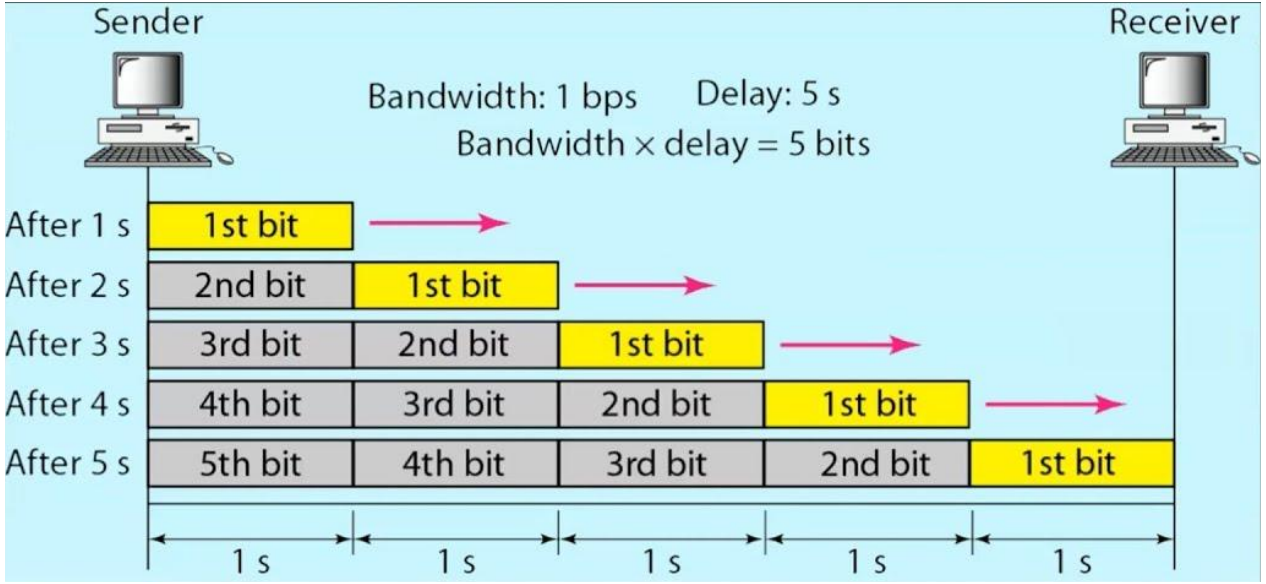
bottleneck link

link on end-end path that constrains end-end throughput

Bandwidth Delay Product

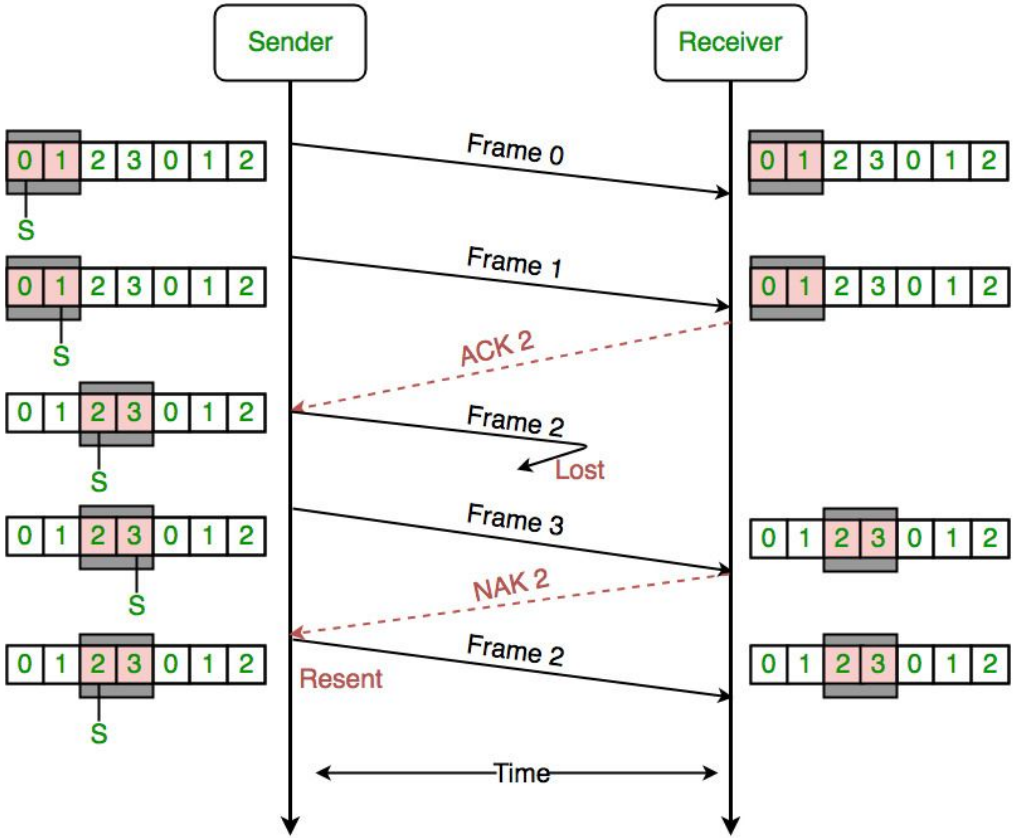
The Bandwidth Delay Product (BDP) is the maximum amount of data (in bits or bytes) that can be "in flight" or in transit within a network link at any given moment.

BDP = Bandwidth * RTT



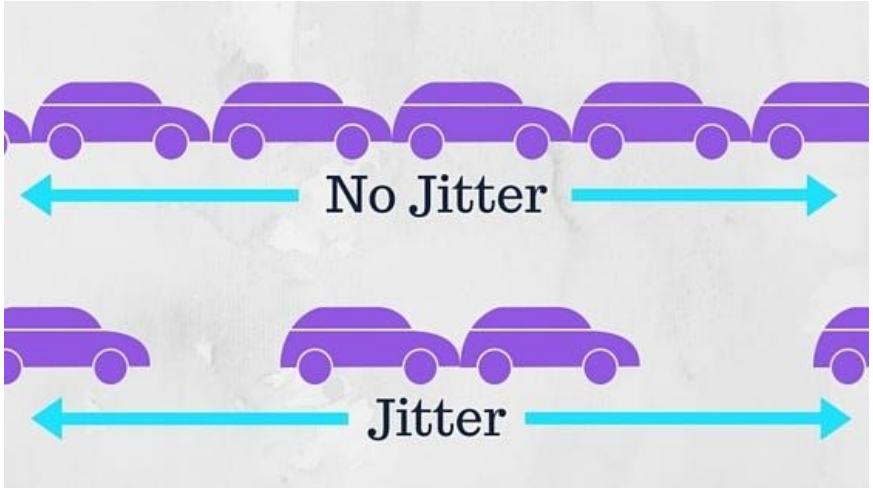
TCP Sliding Window

- Notion of a “window” (cwnd)
- “Windowed” data is sent and response ack awaited
- Window moves forward when ack received for earliest packet
- New data in window is sent out
- TCP dynamically increases window size until congestion occurs



Jitter

Jitter in networking is the variability or inconsistency in latency (delay) of data packets arriving at their destination. While latency is the total time for data transmission, jitter measures how much that time fluctuates, often causing choppy audio, robotic voices, or frozen video in real-time applications.



Problem 1: . How long does it take a packet of length 1,000 bytes to propagate over a link of distance 2,500 km, propagation speed 2.5×10^8 m/s, and transmission rate 2 Mbps? More generally, how long does it take a packet of length L to propagate over a link of distance d, propagation speed s, and transmission rate R bps? Does this delay depend on packet length? Does this delay depend on transmission rate?

Suppose Host A wants to send a large file to Host B. The path from Host A to Host B has three links, of rates R1 = 500 kbps, R2 = 2 Mbps, and R3 = 1 Mbps.

1. Assuming no other traffic in the network, what is the throughput for the file transfer?
2. Suppose the file is 4×10^6 bytes. Dividing the file size by the throughput, roughly how long will it take to transfer the file to Host B?
3. Repeat (a) and (b), but now with R2 reduced to 100 kbps.

Problem 3: Assume n packets of length **L** come into a router at the same time. The transmission rate of the router is **R.** Ignore the processing delay and calculate the average queuing delay.

Socket Programming

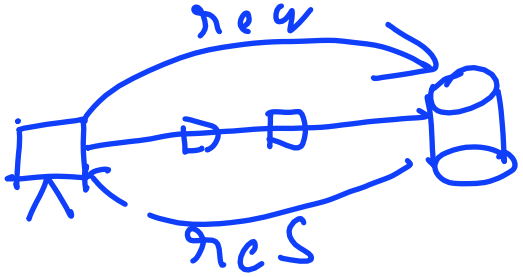
Socket API

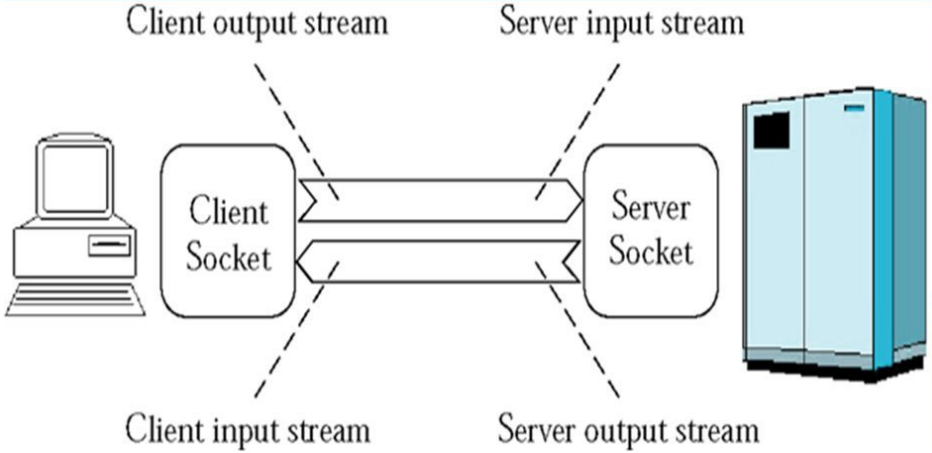
S = (10.1.0.1 , 8000)

Machine

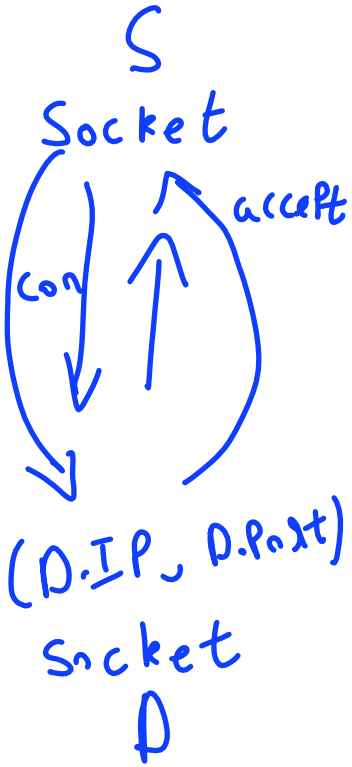
APP

- **Socket:** IP + Port endpoints from which data flows
- Socket API allows writing programs that can send messages across a network (both local and internet)
- Server/client common architecture



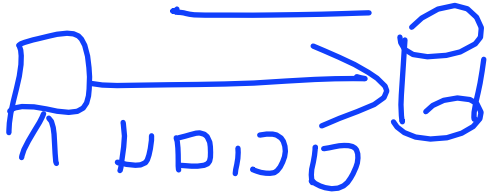


Socket API Methods



Method	Purpose	Client?	Server?
`socket()`	Create a new socket object	Y	Y
`.bind()`	Bind socket to address		Y
`.listen()`	Start listening for new connections		Y
`.accept()`	Accept a new connection		Y
`.connect()`	Connect to a listening socket	Y	
`.connect_ex()`	Connect to a listening socket (raise exceptions)	Y	
`.send()` `.sendall()`	Send a message Send a full buffer	Y	Y
`.recv()`	Receive a message	Y	Y
`.close()`	Close socket	Y	Y

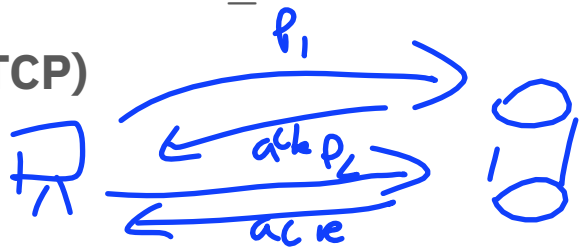
Types of Sockets



User Datagram Protocol (UDP)

- Unreliable and out-of-order

```
socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```



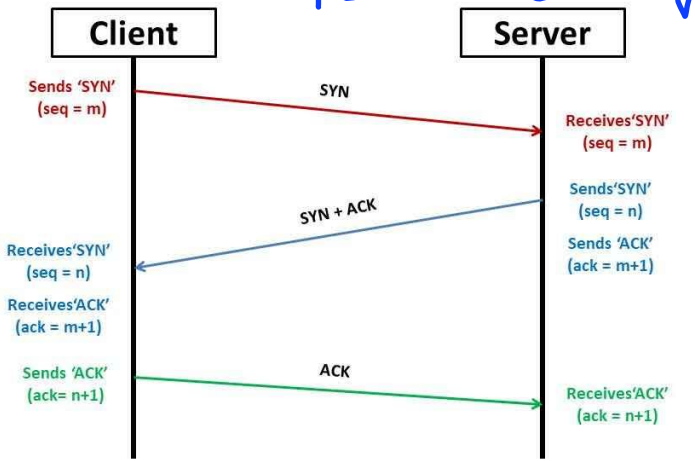
Transmission Control Protocol (TCP)

- Reliable and in-order

```
socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

TCP vs UDP

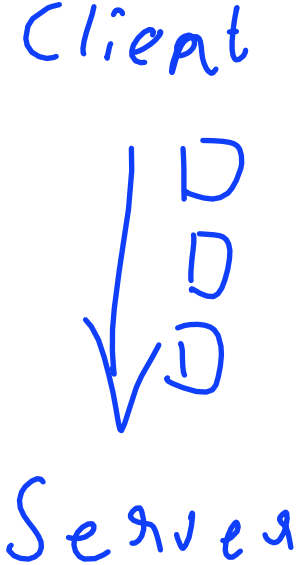
TCP.header → Syn



$P_1, P_2, P_3, P_4, P_5 \Rightarrow$ Mes

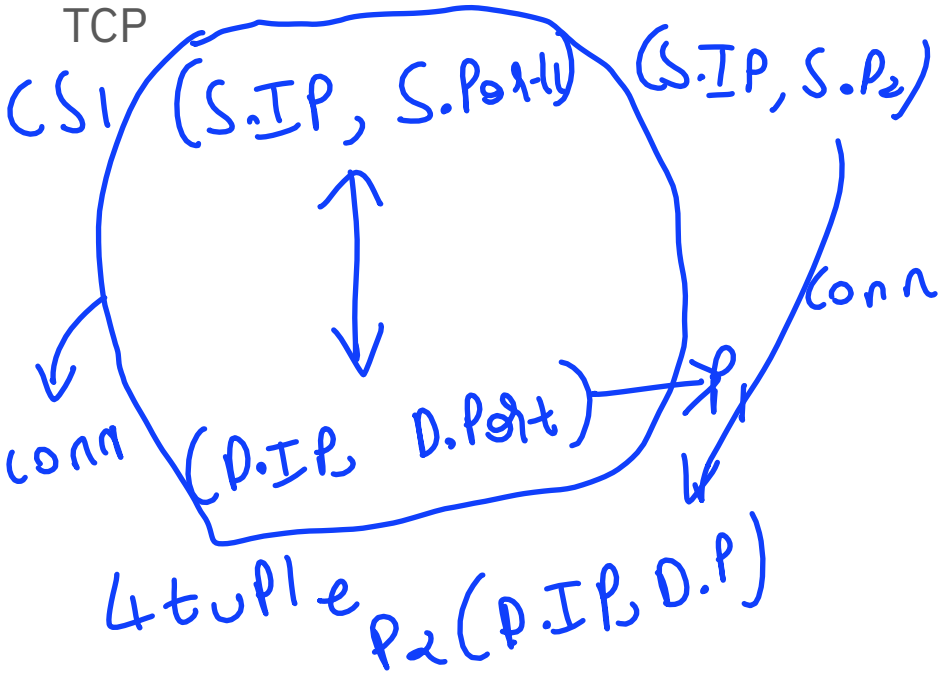
Packet no? $\Rightarrow$ seq = 1 ... n

P_1 = seq = 1 $\leftarrow$ seq = 2



Can two sockets share the same port?

TCP



UDP

Source (S.IP, S.P)

P1 -> (D.IP, D.Port)

P2 -> (D.IP, D.Port)

?

Can two sockets share the same port?

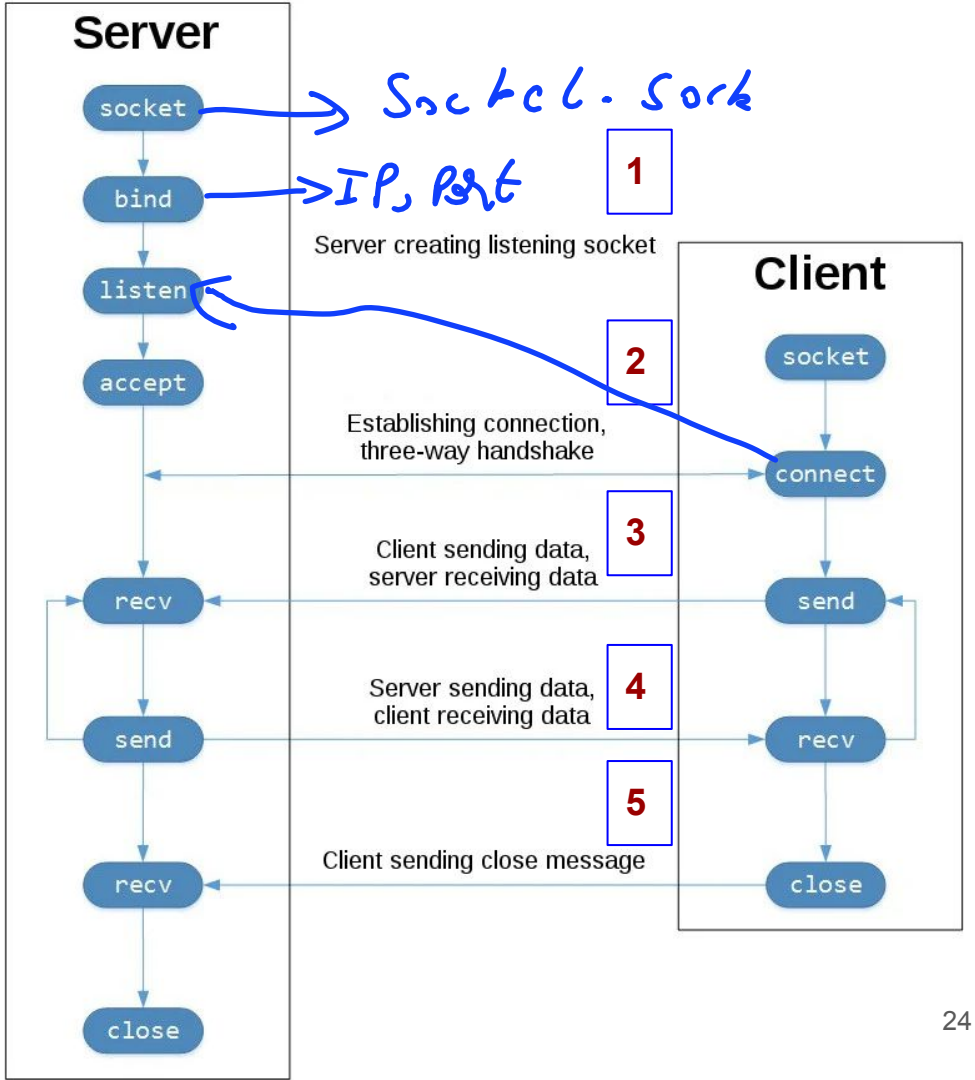
Yes, in case of TCP sockets.

- A connection in TCP is represented as a four-tuple(source IP + source Port + destination IP + destination port)
- Multiple sockets can run on the same port as long as they are connected to different source ports and IP addresses

No, in case of UDP sockets.

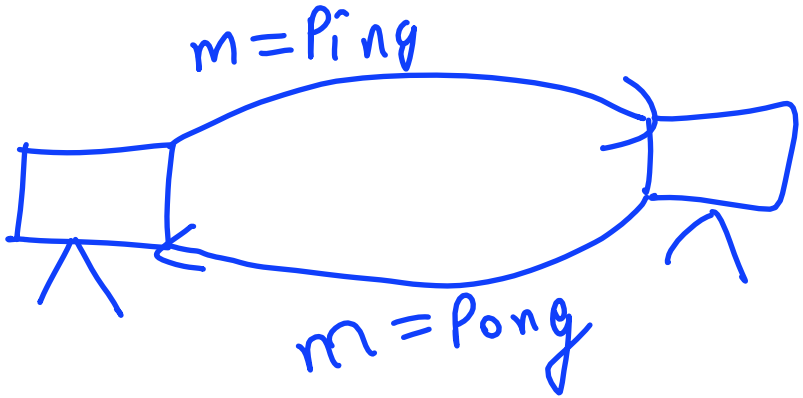
- An incoming packet for a UDP socket is only identified a two-touple (Destination IP address and the port number)
- There is no way to have multiple sockets listening on the same port and determine which socket to deliver the packet to

Sequence of connection flow



Simple ping-pong client-server

- Client sends a ping to server
- Server responds with pong



UDP Ping-pong Server

```
import socket

HOST = '127.0.0.1'
PORT = 5500

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server_socket:
    server_socket.bind((HOST, PORT))

    while True:
        data, addr = server_socket.recvfrom(1024)
        print(f"Received {data.decode()} from {addr}")
```

UDP Ping-pong Client

```
import socket

SERVER_HOST = '127.0.0.1'
SERVER_PORT = 5500

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as client_socket:
    for message_id in range(1, 11):
        message = f"Ping #{message_id}".encode()
        client_socket.sendto(message, (SERVER_HOST, SERVER_PORT))
```

TCP Ping-pong Server

```
import socket

HOST = '127.0.0.1'
PORT = 5500
```

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()

    while True:
        client_socket, (client_addr, client_port) = server_socket.accept()
        data = client_socket.recv(1024)
        if data == b"ping":
            print(f"Received {data.decode()!r} from {client_addr}:{client_port}")
            client_socket.sendall(b"pong")
```

TCP Ping-pong Client

```
import socket

SERVER_HOST = '127.0.0.1'
SERVER_PORT = 5500
```

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((SERVER_HOST, SERVER_PORT))
    s.sendall(b"ping")
    data = s.recv(1024)

print(f"Received {data.decode()!r} from {SERVER_HOST}:{SERVER_PORT}")
```

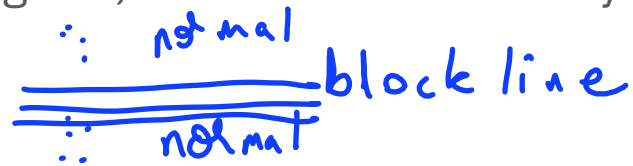
Recap

- UDP Ping-Pong server
  - Server received pings, no means to respond
  - unreliable, out-of-order

- TCP Ping-Pong server
  - Server received pings, could respond with pongs
  - v1: Client sends one ping, server responds with one pong
  - v2: Client sends several pings, server response with equal pongs

Limitations of v1 and v2 servers

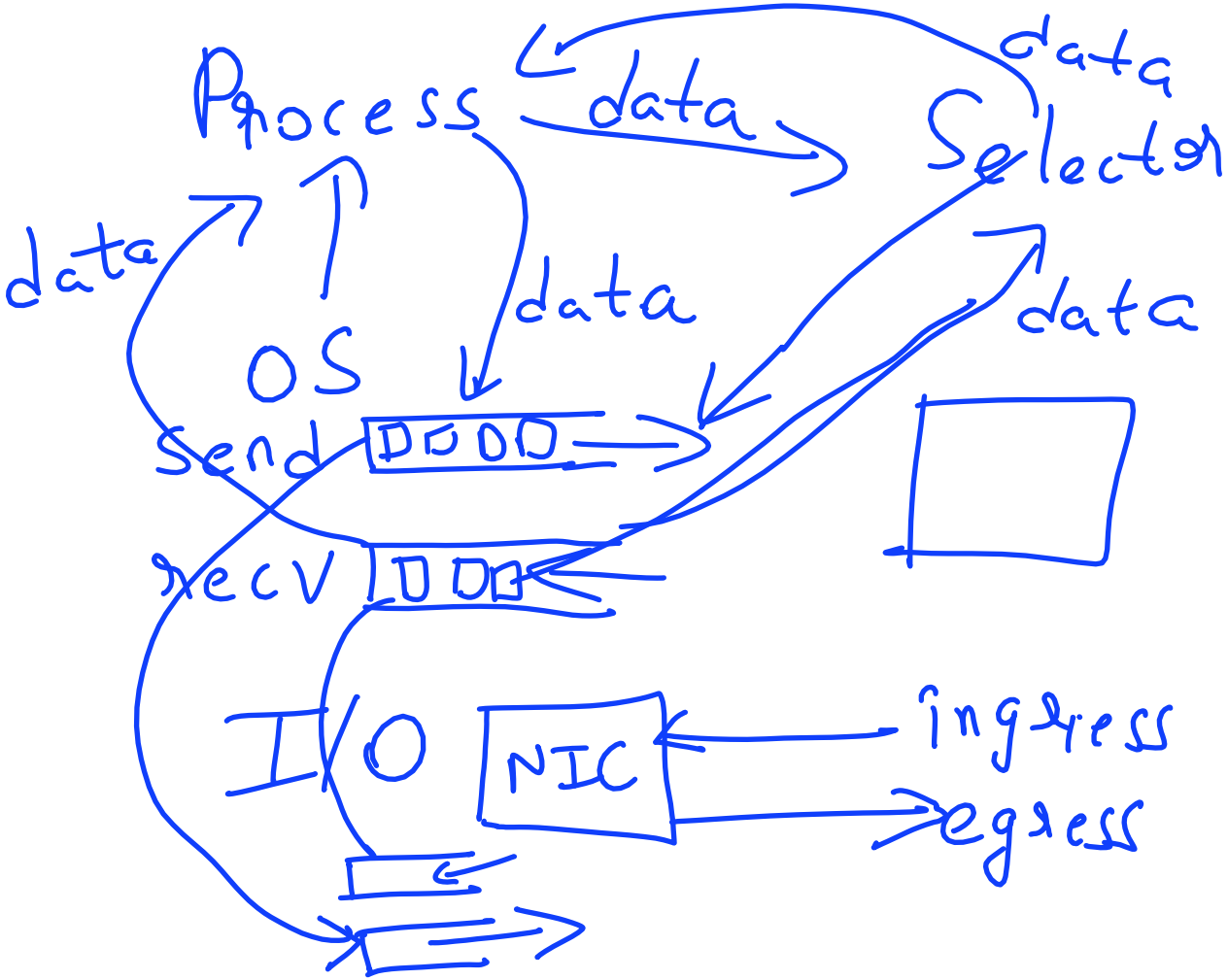
- The server can only serve one client and then exits; same issue for the client
- Even if we want to accept multiple clients and listen multiple times, accept is a blocking call, which means that only one client will be served at a time



How can a server serve multiple clients simultaneously?

Option 1: Selectors

- Provide platform-independent abstractions for I/O multiplexing.
- Event-based, e.g., READ, WRITE
- Register I/O descriptor (e.g., socket object) with selector
  - Also specify events of interest
- Selector notifies when I/O pending on registered descriptors


Process
data
Selector
data
data
data
data
OS
send
recv
I/O
NIC
ingress
egress

TCP Ping-pong Server v3 - Selectors

Mask

0 00 → no load, no write

1 01 → read, no w

2 10 → no read, write

3 11 → read, write

```
import socket, selectors, types

HOST = '127.0.0.1'
PORT = 5500
selector = selectors.DefaultSelector()

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    server_socket.setblocking(False)
    selector.register(server_socket, selectors.EVENT_READ, data=None)

    while True:
        events = selector.select(timeout=None)
        for key, mask in events:
            if key.data is None:
                # accept a new connection
                accept_connection(key.fileobj)
            else:
                # receiving/writing data
                handle_message(key, mask)
```

→ don't wait on OS

mask ⇒ avail events

TCP Ping-pong Server v3 - Selectors

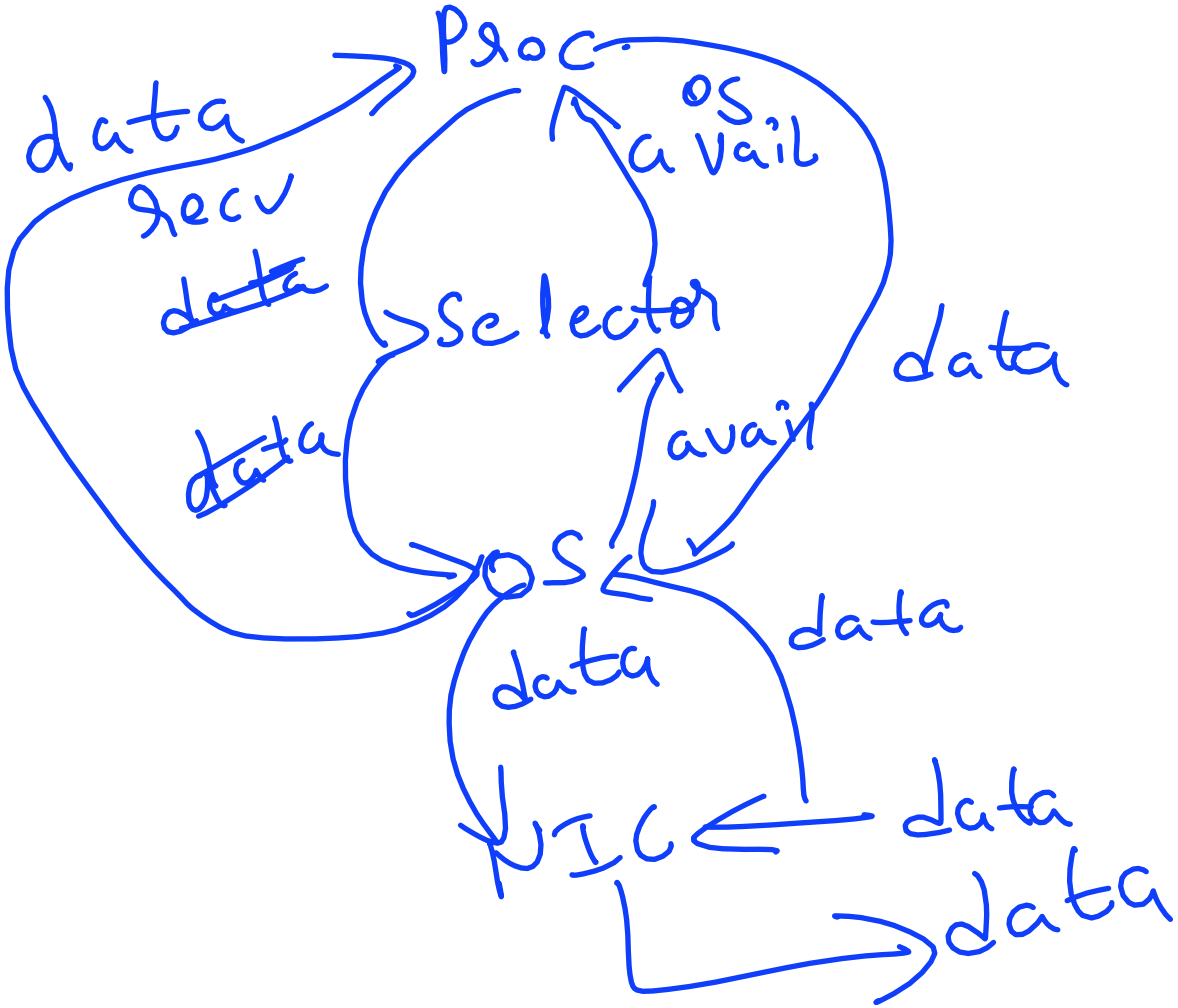
```
def accept_connection(sock: socket.socket):
    conn, (addr, port) = sock.accept()
    data = types.SimpleNamespace(addr=addr, port=port, recv=[], send=[])
    events = selectors.EVENT_READ | selectors.EVENT_WRITE
    selector.register(conn, events, data)

def handle_message(key, mask):
    sock = key.fileobj
    data = key.data
    if mask & selectors.EVENT_READ:
        recv_data = sock.recv(1024)
        if recv_data == b"ping":
            print(f"Received {recv_data.decode()!r} from {data.addr}:{data.port}")
            data.recv.append(recv_data)
            data.send.append(b"pong")
        elif not recv_data:
            selector.unregister(sock)
            sock.close()
    elif mask & selectors.EVENT_WRITE:
        if len(data.send) > 0:
            sock.sendall(data.send.pop(0))
```

new client

registered client

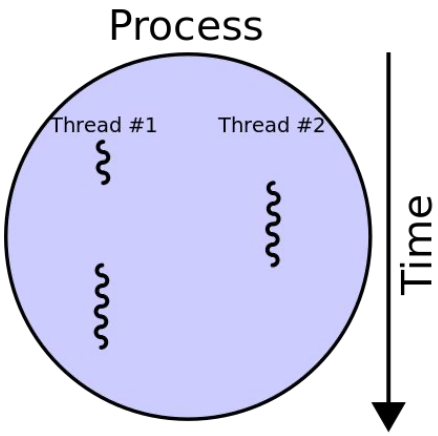
recv P_1 P_2 P

Send P_1, P_2, P_3


Proc
data
os
avail
recv
~~data~~
Selector
data
~~data~~
avail
OS
data
data
NIC
data
data

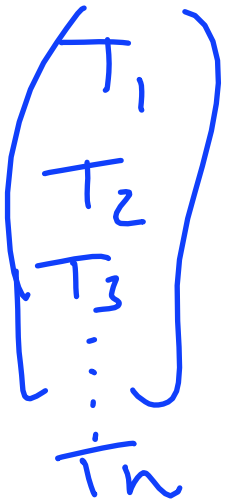
Option 2: Threads

- Smallest sequence of program instructions
- All processes start with one thread
- Multiple threads can be spawned in the same program
- Threads run concurrently
- Python has several libraries
  - multiprocessing
  - threading
  - concurrent



TCP Ping-pong Server v4 - Threads

```
import socket
from concurrent.futures import ThreadPoolExecutor

HOST = '127.0.0.1'
PORT = 5500

def handle_client(socket, addr, port):
    while True:
        data = socket.recv(1024)
        if data == b"ping":
            print(f"Received {data.decode()!r} from {addr}:{port}")
            socket.sendall(b"pong")
        elif not data:
            socket.close()
            break

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    with ThreadPoolExecutor(max_workers=5) as executor:
        while True:
            client_socket, (client_addr, client_port) = server_socket.accept()
            executor.submit(handle_client, client_socket, client_addr, client_port)
```



Server

- Thread1 (S.IP, S.P1) → Pongs
  ← Ping
- Thread2 (S.IP, S.P2) → Pong
  ← Ping
- T3
- T4
- T5

TCP Ping-pong Server v4 - Threads

```
executor.submit(handle_client, client_socket, client_addr, client_port)
```

Arguments:

- `handle_client:` Function to call
- `client_socket, client_addr, client_port:` Arguments to `handle_client`

TCP Ping-pong Server v4 - Threads

```
import socket
from concurrent.futures import ThreadPoolExecutor

HOST = '127.0.0.1'
PORT = 5500

def handle_client(socket, addr, port):
    while True:
        data = socket.recv(1024)
        if data == b"ping":
            print(f"Received {data.decode()!r} from {addr}:{port}")
            socket.sendall(b"pong")
        elif not data:
            socket.close()
            break

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    with ThreadPoolExecutor(max_workers=5) as executor:
        while True:
            client_socket, (client_addr, client_port) = server_socket.accept()
            executor.submit(handle_client, client_socket, client_addr, client_port)
```

New thread executes this code concurrently

This line spawns a new thread

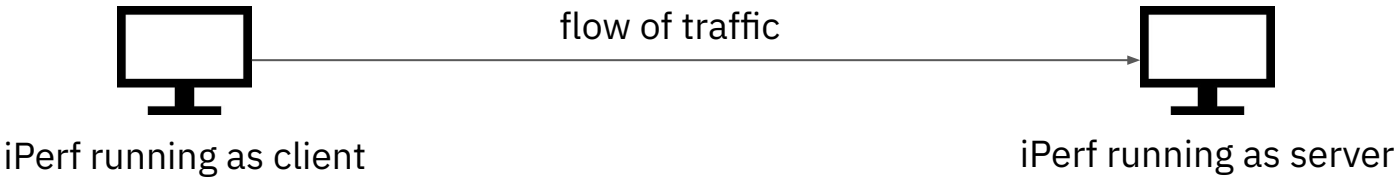
TCP Ping-pong Server v4 - Threads

```
import socket
from concurrent.futures import ThreadPoolExecutor

HOST = '127.0.0.1'
PORT = 5500

def handle_client(socket, addr, port):
    while True:
        data = socket.recv(1024)
        if data == b"ping":
            print(f"Received {data.decode()!r} from {addr}:{port}")
            socket.sendall(b"pong")
        elif not data:
            socket.close()
            break

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    with ThreadPoolExecutor(max_workers=5) as executor:
        while True:
            client_socket, (client_addr, client_port) = server_socket.accept()
            executor.submit(handle_client, client_socket, client_addr, client_port)
```

Original “main” thread goes back into accept blocking call to listen for new connections

Measuring throughput with iPerf

- Tool for active measurements of the maximum achievable bandwidth on IP networks
- Assess quality of the network
- Works in either server mode or client mode
- Client transmits large amount of data to server
- Server times data receipt and reports to client



iPerf Metrics

- **Latency** (response time or RTT): can be measured with the Ping command.
- **Jitter** (latency variation): can be measured with an iPerf UDP test.
- **Datagram loss**: can be measured with an iPerf UDP test.
- **Bandwidth**: measured through iPerf TCP tests.

iPerf Syntax

Server mode:

- `iperf3 -s`

Client mode:

- `iperf3 -c <server_ip>`
- `iperf3 -c <server_ip> --udp`

Public Servers: https://iperf.fr/iperf-servers.php

iPerf Additionals

- -P: parallel connections
- -f: can change the file size display,
- --get-server-output: display the server results on client